



LogicBlock version 1.0.0

⚙️ Setup Guide

LogicBlock requires Game Creator 2 and can't work without it.

Setup

1. [Setup and install Game Creator 2](#)
2. Download and install **LogicBlock | Game Creator 2** from the package manager

Open the **Game Creator Install Window** by going to Game Creator > Install..

Select the Examples located under LogicBlock and click **Install**

The examples can be found in the Plugins > GameCreator > Installs > LogicBlock.Examples@1.0.0 > folder

Updating LogicBlock

When there is a new release, we recommend you to first uninstall the old version.

You can do that by going to the Unity toolbar, Game Creator > Uninstall > LogicBlock

Feedback

For any issues, questions or suggestions, feel free to contact me.

You can get in touch with me on Discord: **lexsern** or send me an email at contolttower@fullscreen.no

Optional Setup

When installing **LogicBlock** from the Package Manager, LogicBlock will automatically check if Game Creator 2 **Inventory**, **Stats**, **Quests**, **Melee**, or **Shooter** modules are installed.

- If any of these modules are detected during installation, LogicBlock will automatically enable full integration support for them.
- If you install any of these modules after installing LogicBlock, you can manually trigger the integration check via the Unity toolbar:
Game Creator > LogicBlock > Detect and Install Integration

If you wish to remove any of these modules, first uninstall LogicBlock and then uninstall the module you no longer want.

You can uninstall LogicBlock by going to the Unity toolbar:

Game Creator > Uninstall > LogicBlock

Afterward, reinstall LogicBlock.

Creating Blocks

To create a new Block asset:

Right-click inside your Project folder → Create > Game Creator > LogicBlock > Block

Adding Instruction and Condition Lists

To create a new Instruction List, click Add Instruction List inside the Block.

To create a new Condition List, click Add Condition List inside the Block.

Recommended Block Setup

Organize your logic by creating Blocks focused on specific areas:

Character Block: Instructions like Jump, Deal Damage, Pickup Item, Interact, Activate Quests, Equip Weapons, etc.

AI Block: Targeting, Following, Patrolling, and other common AI behaviors.

GameObject Block: For game object-related logic, like rotating pickups or environmental objects.

Organizing Lists

Each list has a drag handle allowing you to reorder lists inside a Block to keep your workflow neat and intuitive.

References

Dynamic References allow you to configure behavior dynamically at each instruction without editing the list inside the block directly.

How to Add a Reference:

Open a list inside a Block and click Add Reference.

Set the name and choose the appropriate Reference Type.

Click Create. The reference will appear below the instructions inside the list.

You can click on the reference to expand it and find a Delete button if needed.

Important: References are dynamic and must be set via the instruction that calls the list. They are not saved directly into the Block asset.

Making the instruction use the references:

To make instructions interact with a reference (Example, setting a GameObject active):

Add a GameObject Reference to the list.

Add a “Set Active” instruction to the list.

In the GameObject field of the instruction, select:

LogicBlock > Block GameObject Reference

You'll now be prompted to choose:

The Block the reference comes from.

The List inside that Block.

The specific Reference (Example, the GameObject you added earlier).

If you only have one reference of that type, it will be auto-selected. If you have multiple, click the reference field to see all options and pick the correct one.

This works the same way for all reference types:

Items: use Block Item Reference

Stats: use Block Stat Reference

Booleans, Numbers, Strings, etc., each have their own reference type selectors inside compatible instructions.

This system allows your instructions to stay completely modular and flexible, supporting centralized logic with input driven variation.

Playmode Button

Each list features a Play button available in Play Mode to instantly run the entire instruction list for quick testing.

If the list uses dynamic references, these must first be set by running the instruction normally. After that, the Play button reuses the last valid reference values.

Edit Button

Click the Edit button to expand the list's customization options: rename the list and assign an icon

The icon selector pops up a window where you can pick from Game Creator 2's icon library and module icons, along with customizable colors.

A search field helps quickly find the icon you want.

Delete Button

Click the Delete button to remove a list. A confirmation popup prevents accidental deletions.

Duplicate Button

Click the duplicate button to duplicate a list.

Performance

LogicBlock optimizes editor performance by loading instruction and condition lists only when expanded. This keeps the Unity editor fast and responsive, even with many lists in a Block.

Self and Target

Inside a Block, Self and Target references behave exactly as if the instructions were placed inside the trigger itself, preserving expected context seamlessly.

Running Block Lists

LogicBlock comes with two built-in instructions:

Run Instruction Block List

Run Conditions Block List

These let you trigger reusable logic and condition checks stored in a Block asset.

Running an Instruction or Condition List

To execute logic from a Block, use the appropriate instruction inside a Trigger or Action:

Use Run Instruction Block List if you want to run a list of instructions.

Use Run Conditions Block List if you want to evaluate a condition list.

1. Select the Block

In the instruction, use the Block field to assign the Block asset that contains the list you want to run.

Once selected, a dropdown will appear showing all the Instruction or Condition Lists available in that Block. Choose the list you want to execute.

2. Set the Target

There's a Target dropdown which determines who or what will be the target of the executed instructions.

By default, this is set to Target, which is standard Game Creator 2 behavior.

However, you can override this and specify a custom target (such as Self, a Game Object, or a dynamic reference), depending on your needs.

3. Instancing (Optional but important)

You'll see an option called Instance Block.

Enable this if you plan to run the same Block List simultaneously from multiple sources.

For example:

You have five GameObjects rotating at the same time.

You trigger the same logic across several enemies or pickups at the same time.

If instancing is not enabled and you run the same list multiple times simultaneously, you may experience unexpected behavior or interference between calls.

Handling Dynamic References

If the selected list contains Dynamic References, all of them will appear as individual fields inside the instruction.

You must assign values for each reference here, these will be passed into the Block when the list runs.

For example:

If the list expects a GameObject and an Item, you'll get fields where you can assign those values before running the instruction.

This allows you to re-use a single logic list across your entire game while customizing its behavior through references on a per-call basis.

⚠️ You can also leave reference fields empty if your logic doesn't depend on those values. Any instructions relying on an unassigned reference will behave accordingly (Example, do nothing, fail silently, or use defaults, depending on how they're built).

💡 Tip: Collapse for Performance

After you've finished configuring the instruction, setting references, and selecting the block, it's recommended to collapse the instruction in the Inspector.

This prevents Unity from reloading and loading the full references list every time you select the GameObject, helping to keep the editor fast and responsive, especially in larger projects or when using many LogicBlock calls in the same GameObject

Handles in LogicBlock

Handles are not a native GC2 property type by default. LogicBlock adds support for Handles as a custom property type.

LogicBlock includes a special Attach Prop (Handle) instruction that lets you select Handles dynamically via property.

🔗 LogicBlock supports referencing the following Game Creator 2 property types: 🔗

GameObject

String

Number

Boolean

Color

Vector3

AnimationClip

AudioClip

Material

Sprite

Texture

Handle

Scene

Scale

Rotation Direction

Item

Runtime Item

LootTable

Quest

Shooter Weapon

Melee Weapon

Shield

Skill

Attribute

Formula

Stat

Status Effect

Troubleshooting

LogicBlock uses a highly dynamic custom visual interface to provide a powerful and intuitive editing experience. However, Unity's UI system (VisualElement) can occasionally struggle with complex editor drawers like the one LogicBlock uses.

In rare cases, you might see fields display as "Missing Block" or "Missing List" in the instructions Run Block List / Run Condition Block List unexpectedly. This visual glitch can occur during:

Moving the inspector window

Opening a second inspector

Rapidly switching between GameObjects

Unity refreshing the layout mid-frame

 Good news:

This is a purely visual issue. Your LogicBlock data is safe and unaffected. The instruction lists continues to function correctly, and no references is lost.

To fix it:

Simply deselect and reselect the GameObject in the hierarchy. The editor UI will refresh, and everything will display correctly again.

 Important:

Using two inspector windows showing the same GameObject is not recommended. Unity does not sync inspectors in real-time, which can result in data loss or unexpected behavior. To stay safe, edit LogicBlock instructions using only one inspector at a time.

Restart Instructions

When creating instructions that run over time, such as rotating a GameObject continuously, you might want to restart the instruction to loop the instruction.

Important: Always add a "Wait 1 Frame" instruction before restarting the instruction list. This small delay allows Unity to process the current frame fully and prevents the editor or game from freezing.

If you restart the instruction without waiting a frame, it can cause an immediate loop that overloads Unity's update cycle, resulting in the editor or game becoming unresponsive.